

PROJECT REPORT

A practical guide to what the app does and how it works

FoodShare connects surplus food donors with NGOs and community organizations. Its purpose in this project is twofold: demonstrate a useful donation-to-delivery workflow, and make the C data structures behind that workflow visible and understandable.

PROJECT AT A GLANCE

MISSION

Reduce food waste; route surplus meals to people who need them.

WEB

React + TypeScript interface, hosted as a Vercel SPA.

CORE

C data-structure library and a small JSON HTTP server.

API

Native C service in a Render Docker container.

Try the public demo

Website: <https://smart-food-donate.vercel.app>

Live C API health check: <https://smart-food-donate-c-api-justanoob777-c0j9.onrender.com/api/health>

Important: this is a public learning demo

It uses demo credentials, client-side role selection, a public API, and volatile in-memory data. Do not enter personal or sensitive information. The free API may sleep while idle and data resets when the service restarts.

1. What the project is

Purpose, people, and the parts that make one donation workflow

The application models a community food-sharing process. A donor lists surplus food, an NGO requests some of it, a coordinator manages dispatch, and the delivery advances through a small sequence of milestones. The project is also a C-learning tool: the server's operations are backed by actual stack, queue, deque, heap, linked-list, balanced search-tree, and graph code.

Who does what

Participant	Typical action	Where to try it
Donor	Describe food, quantity, pickup location, and expiry.	/donate or /c-demo
NGO	Browse available food and request a serving quantity.	/browse or /c-demo
Coordinator / admin	Review requests, reorder dispatch, approve the next pickup.	/dashboard, /ds, /c-demo
Volunteer / tracker	Advance a request through pickup and delivery steps.	/track
Learner	Inspect live structures, search, and compare graph traversals.	/ds and /c-demo

The main pages

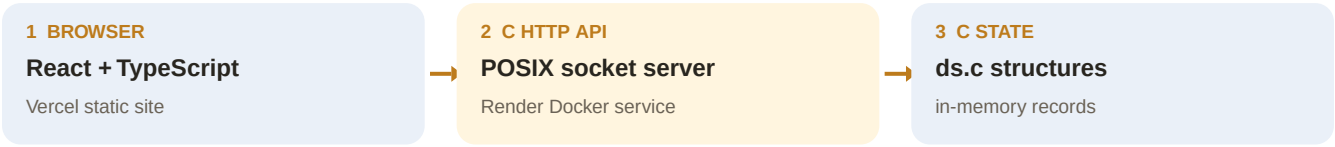
Route	What it is for
/	Public introduction and entry point.
/auth	Local demo sign-in; select donor, NGO, or admin.
/dashboard	Role-aware summary of food, requests, and activity.
/donate	Create a food donation.
/browse	Filter/search donations and request food.
/track	Select requests and move their delivery progress.
/ds	Live operations explorer, organized by structure.
/c-demo	Direct C API workbench with forms and structure diagrams.
/about	Mission, project rules, and technology overview.

A note about the word 'live'

The API data shown by /ds and /c-demo is held by one running C server, so visitors to the public demo share that in-memory board. A user's demo sign-in is stored in that browser; it is not a secure account.

2. How a user action becomes C work

From a button in the browser to a concrete algorithm and back



Browser-side C is not a replacement for the backend
A second route uses the same C algorithms compiled as ds.wasm in the browser. Product pages can use this module for local algorithm traces; the live records and shared board still come from the C HTTP API.

Donation-to-delivery in four moves

Step	Input and checks	C structures that change
1. Offer food	Title, location, positive serving count, and future expiry.	BST indexes its ID; min-heap records expiry priority; stack records activity; graph is rebuilt.
2. Request food	NGO/admin role, known available donation, valid quantity.	BST verifies the donation; FIFO queue records arrival; deque builds dispatch order; graph and stack update.
3. Dispatch	Admin may move a pending request to front/back, then approve the current dispatch-front request.	Deque expresses handling priority; FIFO arrival order remains a separate view.
4. Deliver	NGO/admin advances the request one milestone.	Linked list represents route progress; donation status and stack activity update.

Two ways to execute the C algorithms

- The deployed experience uses a browser UI calling the native C HTTP API. This is the shared server-side state that multiple visitors see.
- The repository also ships public/ds.wasm, compiled from the C structure library and its WebAssembly adapter. Product pages use it for some local structure work and traces; this browser copy is not the shared records database.
- The /ds operations screen currently reads and mutates live API structures through HTTP. Its pictures are views of the API state, not a static mock.

Simple mental model
React gathers input and draws results. The native server validates important workflow rules and calls ds.c. The C structures decide order/search/traversal. The server serializes results as JSON for the page.

3. The donation and pickup workflow

What gets validated, which state changes, and what users see

A. Donating food

On /donate, the form asks for a name, food category, serving quantity, pickup location, expiry time, and optional notes. On /c-demo, the smaller form focuses on food name, donor/business name, pickup location, quantity, and minutes until expiry.

- The browser checks that required values exist and are plausible; the C endpoint independently requires title, location, positive quantity, and positive expiry.
- C assigns a donation ID beginning at 1001. It inserts the ID into the self-balancing BST and inserts (ID, expiry minute) into the min-heap.
- The donation is marked AVAILABLE. The server rebuilds the donor/hub graph and pushes an action code onto the activity stack.
- The donations endpoint copies the heap and extracts from the copy, so it can return soonest-expiring entries first without consuming the live heap.

B. Requesting food

On /browse or /c-demo, an NGO/admin chooses an available donation and a quantity. The native API checks the role value, quantity, donation ID, availability, and that the requested servings do not exceed the amount offered.

- A request is assigned an ID starting at 5001. The donation changes from AVAILABLE to CLAIMED, and the request begins as PENDING.
- The request ID is enqueued at the FIFO rear. The dispatch deque is rebuilt separately so urgent work can appear at the front while the queue still shows arrival order.
- The graph adds the NGO-to-donor connection for the requested donation; an activity code is pushed to the stack.

C. Admin dispatch and delivery

- By default, pending donations expiring within 120 minutes are treated as urgent. An admin can explicitly pin a request to the front or return it to the routine back.
- Approval normally applies to the current front of the dispatch deque. The server removes that request from the pending FIFO while retaining other arrivals in their original order. A separate /api/requests/claim endpoint illustrates removing the oldest FIFO item.
- Delivery progresses through Request Received, Assigned to Volunteer, Picked Up, Out for Delivery, and Delivered. On pickup, the donation becomes COLLECTED; on completion it becomes DISTRIBUTED.
- The route list is rebuilt to contain milestones reached so far. The tracker shows progress, the route, the request status, and action feedback.

State labels

AVAILABLE means an offer may be requested. CLAIMED means a request exists. COLLECTED is set once delivery reaches the pickup stage. DISTRIBUTED is the completed state.

4. Three ways to organize requests and actions

Stack, circular queue, and double-ended queue

Stack - newest activity on top

A stack is last-in, first-out (LIFO). FoodShare pushes an action code after important events such as donation creation, request creation, approval, or route advancement. The history endpoint reads from the newest action down, which is why the most recent event appears first.

Concept	In FoodShare
Push	Record a new action at the top.
Pop / peek	Remove or inspect the top item; used directly in the C lab operations.
Capacity	64 integer action entries.
Cost	Push, pop, and peek are $O(1)$.
Code	<code>stack_init</code> , <code>stack_push</code> , <code>stack_pop</code> , <code>stack_peek</code> in <code>c-backend/ds.c</code> .

Circular queue - oldest request first

A queue is first-in, first-out (FIFO). New request IDs join at the rear; the front is the oldest. The C implementation stores items in a fixed array and wraps head and tail using modulo arithmetic. This reuses freed slots instead of shifting every item.

Operation / detail	Effect
<code>enqueue(id)</code>	Place at tail; advance tail modulo 64.
<code>dequeue()</code>	Return head; advance head modulo 64.
<code>front()</code>	Read the next request without removing it.
Capacity / cost	64 entries; <code>enqueue/dequeue</code> are $O(1)$.

Deque - dispatch from either end

A deque (double-ended queue) supports insertion/removal at both ends. The live dispatch lane is rebuilt from pending requests: urgent items go nearer FRONT and routine ones stay toward BACK. Admin front/back overrides are recorded and ranked so the lane is deterministic.

- Automatic urgency threshold: donation expires within 120 minutes.
- The dispatch picture and the FIFO queue answer different questions: 'what should be handled next?' versus 'who asked first?'
- Capacity is 64 entries. End operations are $O(1)$; this server rebuilds the lane from the pending queue after workflow changes.

5. Expiry, lookup, and delivery route

Priority queue, balanced search tree, and singly linked list

Priority queue - earliest expiry first

The donation urgency structure is a binary min-heap. Each heap item stores a donation ID and its expiry minute. In a min-heap, every parent priority is no larger than its children; therefore the root is the earliest expiry. The heap is not a fully sorted array, so the endpoint repeatedly extracts from a copy to produce a completely ordered response.

Operation	What it does	Cost
pq_insert(id, expiry)	Add a donation; sift upward to restore heap order.	$O(\log n)$
pq_peek()	Inspect the earliest-expiring item.	$O(1)$
pq_extract_min()	Remove the root, move the final item to root, sift downward.	$O(\log n)$
Limit	Fixed array; 64 items. Browser filters status to AVAILABLE.	Capacity

Donation ID index - AVL-balanced BST

The project calls this a BST, but the implementation also stores subtree heights and performs rotations after insertions/deletions. It is therefore a self-balancing AVL-style binary search tree: smaller IDs go left, larger IDs go right, and height is kept balanced. This prevents newly assigned increasing IDs from degenerating into a long chain.

- Search compares one node at a time and follows left/right pointers. The server reports depth with root at depth 0; a missing key returns depth -1.
- Inorder traversal lists IDs in ascending order. The API also returns the actual root/left/right links used to draw the tree.
- The backend node pool has a capacity of 128 donation IDs. Search, insert, and delete are $O(\log n)$ when balanced.

In /c-demo, the search box accepts an ID or exact case-insensitive food title, donor name, or location. The browser resolves text to one donation; it then asks the real C BST endpoint to look up that numeric ID. No match and ambiguous matches are explained instead of silently picking one.

Linked list - delivery progress

A singly linked list stores a value plus the index of the next node. HEAD identifies the first step, each next link advances the route, and -1 means the end. The C list uses a fixed node pool/free list (64 nodes) and keeps both head and tail, so appending a route step is $O(1)$. Searching/removing a value is $O(n)$.

For a request, the route list contains progress milestones reached so far. The browser draws the sequence and labels its pointers; the server keeps the request's current step and rebuilds the list as the delivery advances.

6. Distribution graph and traversal

Represent relationships, then compare BFS with DFS

The graph models the food-sharing network using numbered nodes and undirected edges. Node 0 is the Food hub. A donor/business node is linked to the hub when a donation is recorded. When an NGO requests that donation, its NGO node is connected to the donor node. Repeated donor IDs and NGO references reuse their existing node rather than creating duplicates.

Graph data model

Graph concept	How this project uses it
Vertex (node)	Food hub, donor/business, or NGO.
Edge	A real relationship: hub-to-donor, then donor-to-requesting NGO.
Representation	Undirected adjacency matrix, at most 16 nodes.
IDs and labels	IDs identify positions in the current graph; labels show business/NGO names.
Refresh	Network is rebuilt from current donation/request records.

BFS versus DFS

Traversal	Plain-language rule	Typical effect
BFS	Visit neighbors first, using a queue to keep the next frontier.	Explores nearby links in breadth/levels.
DFS	Follow one link as far as possible, then backtrack.	Explores one branch deeply before others.

On `/c-demo`, choose any node as the start and compare both orders against the same node labels. On `/ds`, the graph panel can run either traversal. The returned order is the C algorithm's result, not an ordering guessed by the drawing code.

Graph size is intentionally small

The C graph supports up to 16 nodes. This keeps a simple adjacency matrix appropriate for the learning demo. Traversal is $O(V^2)$ with this matrix representation; the project is not intended to be a city-scale routing engine.

What the visual cues mean

- Numbered circles are stable IDs for that graph snapshot; text labels tell you which person or organization each node represents.
- Lines are adjacency edges. Highlighting marks vertices included in a traversal.
- BFS and DFS can visit the same connected set in a different order; the order depends on the selected start and neighbor scanning order.

7. How to explore the application

A beginner-friendly route through the public demo and learning screens

Recommended first visit: /c-demo

Try this	What to watch
Open the public demo	The page calls the live health endpoint; the badge changes to C API connected once it responds.
Click Load complete example	Adds sample businesses, donations, pickup requests, and one delivery step. The action is designed to be safe to repeat.
Add one donation yourself	Enter food, a donor/business name, pickup location, servings, and expiry minutes.
Request a pickup	Sign in as NGO or admin first; select an available donation and quantity.
Inspect each structure	Stack recent actions; queue arrival order; deque dispatch order; heap expiry order; BST tree; linked route; graph links.
Search the BST	Type a food title, donor, place, or numeric ID. The text resolves to an ID; C reports found/not found and depth.
Compare BFS + DFS	Choose a graph start node and compare the traversal sequences.

Then visit /ds

The operations page is a live dashboard with summary counts and panels for overview, activity stack, request queue, dispatch deque, expiry priority queue, delivery list, BST index, and distribution graph. Refreshing reads the same C server's current state. The navigation can open directly on a structure, for example `/ds?tab=bst`.

Role-based demo sign-in

Demo role	Username / password	Purpose
Donor	donor / donor123	Offer surplus food.
NGO	ngo / ngo123	Request food and advance deliveries.
Admin	admin / admin123	Reorder/approve dispatch and use all demo workflows.

The browser stores the selected profile in `localStorage`. These are fixed public demo accounts, not private user accounts. Some screens require the NGO/admin role to submit a pickup; /c-demo can also be explored without signing in.

Example names

Donor/business names are user-entered donation data (or sample hotels/cafes from the example loader). Graph labels distinguish Donor: ... from NGO: ... so the network is easier to read.

8. Code map and HTTP interface

Where the behavior lives in the repository

Path	Responsibility
src/main.tsx	React Router routes, auth wrappers, lazy page loading.
src/pages/CDemo.tsx	Direct C API forms, sample loader, diagrams, text-to-ID BST search, BFS/DFS comparison.
src/pages/DSLlab.tsx	Live operations dashboard; reads server snapshots and issues administrative operations.
src/pages/Donate.tsx, Browse.tsx, Track.tsx	User workflow forms; use C API adapter and, on product interactions, WebAssembly C engine operations.
src/lib/c-api.ts	Frontend adapter translating application queries/mutations to HTTP endpoints.
src/lib/c-engine.ts, src/hooks/use-c-engine.ts	Typed wrapper for the C WebAssembly exports and engine lifecycle.
c-backend/ds.h, ds.c	Fixed-capacity data structure declarations and implementations.
c-backend/server.c	Native C POSIX HTTP server, JSON parsing/validation, endpoint-to-structure workflow.
c-backend/wasm_api.c, public/ds.wasm	C-to-WebAssembly operation ABI/trace log and the compiled browser module.
c-backend/tests.c, Makefile	Native structure tests and build/run commands.
c-backend/Dockerfile, render.yaml	Docker build and Render Blueprint for the C API.
vercel.json	Vite build output and SPA route fallback for Vercel.

Important API endpoints

HTTP endpoint	Purpose / structure
GET /api/health	C engine and structure counts; Render health check.
GET /api/donations; POST /api/donations	Expiry order via heap; create validates, BST insert + heap insert.
GET /api/requests; POST /api/requests	Pending arrival order; validate and enqueue a request.
GET /api/dispatch; POST /api/dispatch/reorder	Deque ordering; admin can move a pending item front/back.
POST /api/requests/approve or /claim	Approve dispatch-front item, or remove oldest FIFO item.
POST /api/requests/advance	Advance delivery status and linked route.
GET /api/history; /api/bst; /api/route	Stack history; balanced ID tree; linked route.
GET /api/graph/bfs?from=N; /dfs?from=N	Graph relationships and C traversal order.
GET /api/stats; /api/stats/weekly	Dashboard totals and donation summaries.

9. Build, testing, and hosting

How it runs locally and what the public deployment depends on

Local development

The native server uses POSIX sockets, so the documented Windows workflow is Ubuntu under WSL2. Install Node.js/npm and gcc/make in the Linux environment, clone the repository there, then run the C API and React app in separate terminals.

Command	Purpose
npm ci	Install the web dependencies from the lockfile.
make api (project root)	Build/run the native C HTTP server on port 8080.
npm run dev	Start Vite's local frontend, usually at http://localhost:5173 .
cd c-backend && make test	Run native data-structure tests.
npm run build	Type-check and produce the static web build in dist/.

Current validation record

The C test suite reports 385 passing checks across six tested structures (stack, queue, priority queue, linked list, BST, and graph). The native C server builds. The latest React production build and ESLint check for CDemo.tsx passed before deployment.

Public hosting arrangement

- Vercel serves the static React/Vite frontend at <https://smart-food-donate.vercel.app>. `vercel.json` sends application routes back to `index.html` so direct routes such as `/c-demo` work.
- Render runs the native C API container at <https://smart-food-donate-c-api-justanoob777-c0j9.onrender.com>. Its `/api/health` endpoint returns `engine: c` and structure counts.
- Vercel's build variable `VITE_C_API_URL` points to the Render URL. Frontend API requests stay in the page and call the API over HTTPS; friends do not need to open Render's dashboard.
- The free Render service sleeps after 15 minutes without inbound traffic and may take about a minute to wake. The first request may wait; the visitor remains on the FoodShare site.
- Backend arrays/heaps/trees are in RAM, not a database. Restart, redeploy, or free-service spin-down clears demo records.

10. Limits, safety, and glossary

What the project demonstrates - and what it does not promise

Know before sharing

- This is a learning prototype, not a production food-safety or logistics service.
- Demo passwords are public and roles live in browser storage. The backend accepts a role field from requests for the demonstration; this is not server-side identity verification or authorization.
- The API is publicly reachable and its CORS policy allows browser access. Anyone with the endpoint can send requests; the demo board is shared among visitors while the server is running.
- Data is volatile: do not use it for records that must persist. Any restart/sleep cycle can clear records.
- The heap, stack, queue, and deque use fixed capacities (64 entries each); the tree pool has 128 nodes; graph has 16 vertices; the server donation/request arrays each have 128 slots.
- The project does not calculate real driving routes or verify food safety. The route diagram is illustrative; the graph is a small relationship/traversal model.

Mini glossary

Term	Meaning here
API	HTTP interface used by the browser to ask the C server for data/actions.
BST / AVL	Ordered binary tree for donation IDs; rotations keep height balanced.
Deque	Double-ended queue. Both FRONT and BACK can be reordered.
FIFO / LIFO	First-in first-out / last-in first-out.
Heap	Compact tree-in-array; min-heap root is the smallest expiry priority.
Node / edge	Graph entity / connection between entities.
$O(1)$, $O(\log n)$, $O(n)$	How operation work scales: constant, logarithmic, or linear.
WebAssembly (WASM)	Portable compiled module; here it runs the C algorithms inside a browser.

Further reading in the repository

Start with README.md for setup and project structure; c-backend/README.md for the C library, endpoint map, and complexity notes; c-backend/ds.h for capacities/API signatures; c-backend/ds.c for the algorithms; c-backend/server.c for endpoint validation and workflow transitions; and c-backend/tests.c for boundary and correctness examples.

The central idea

The structures are not decorative: the live server uses them to order food, retain arrival order, prioritize dispatch, index IDs, record recent activity, build delivery progress, and traverse donor/NGO relationships. The interface makes those state changes visible.